

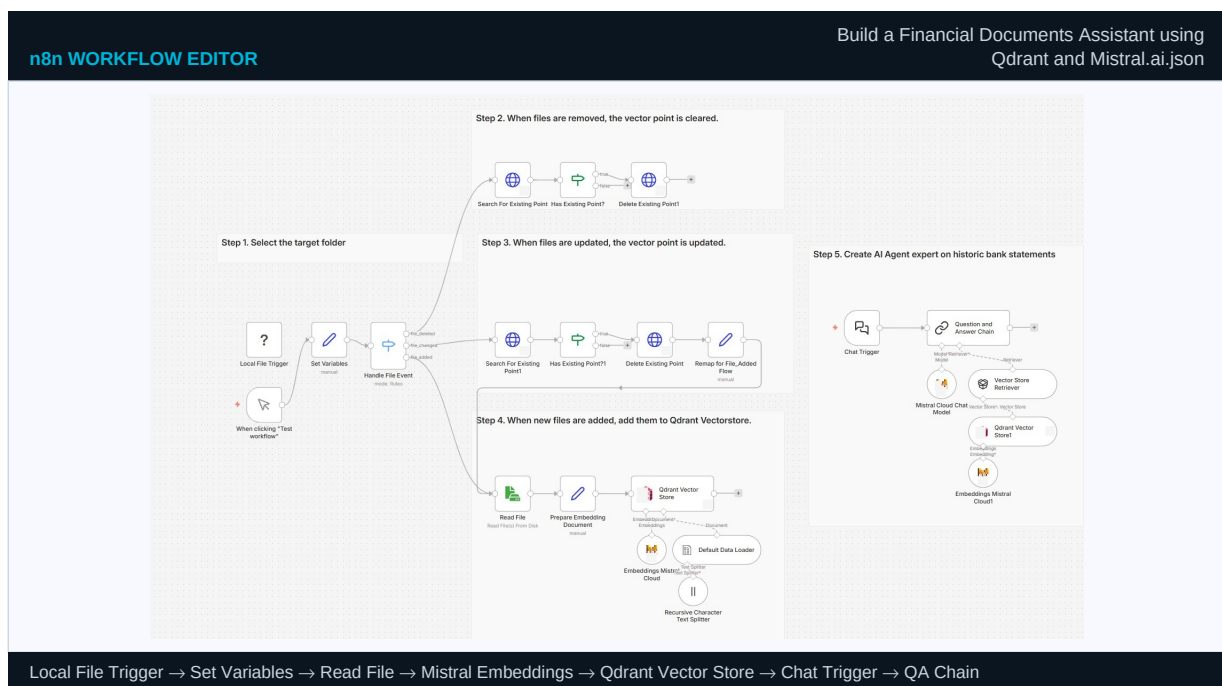
#07 Financial Documents RAG Assistant

RAG / Financial
Intelligence*Chat with your entire financial document library — cited answers in seconds*

BUSINESS PROBLEM

Analysts spend hours manually reading PDF annual reports to find specific figures. This production RAG pipeline watches a folder for new financial PDFs, auto-ingests them into Qdrant with document metadata, and enables precise natural language Q&A; with exact page citations.

n8n CANVAS — actual workflow screenshot



INPUT / OUTPUT — exact n8n JSON format

INPUT — Chat Trigger — analyst question (after PDF ingestion)

```
{
  "question": "What was the gross margin in FY2023 and how does it compare to FY2022?",
  "session_id": "analyst-session-001",
  "company": "AAPL",
  "doc_type": "annual_report"
}
```

OUTPUT — QA Chain response — cited answer with page references

```
{
  "json": {
    "answer": "Gross margin in FY2023 was 44.1% vs 43.3% in FY2022, an improvement of 80 basis points driven by higher-margin Services revenue mix increasing from 19.8% to 22.2% of total revenue.",
    "sources": [
      {"file": "AAPL_Annual_FY2023.pdf", "page": 41, "section": "Income Statement"},
      {"file": "AAPL_Annual_FY2023.pdf", "page": 31, "section": "MD&A"}
    ],
    "confidence": 0.94,
    "model": "mistral-medium"
  }
}
```

STEP-BY-STEP — what each node does

#	WHAT HAPPENS	PRACTICAL DETAIL
1	■ Local File Trigger — new PDF detected	Watches /documents/financial/ folder. Fires on any new or modified PDF automatically — no manual trigger needed.
2	■ Set Variables — extract metadata	Parses filename: AAPL_Annual_FY2023.pdf → {ticker:'AAPL', doc_type:'annual_report', year:'2023'} as Qdrant payload.
3	■ Read File — load PDF binary	Reads raw PDF bytes into n8n binary field. Handles up to 500-page documents without timeout.
4	■ Document Loader — parse PDF text	Extracts text preserving page numbers. Financial tables (P&L, balance sheet) converted to structured text rows.
5	⌘ Recursive Splitter — 1000-char chunks	Splits with 200-char overlap. Section headers (MD&A, ITEM 1A) prepended to each chunk for retrieval context.
6	■ Mistral Embeddings — vectorise chunks	mistral-embed model generates dense vectors. Metadata tagged: {company, year, page, section, doc_type}.
7	■ Qdrant — upsert with idempotency	IF node checks if vector point exists. Deletes old before inserting new. Prevents duplicate embeddings on re-ingest.
8	■ Chat Trigger — analyst question	User asks a natural language question. Trigger passes to the QA Retrieval Chain for answer generation.
9	■ Vector Retriever — semantic search	Question embedded → top-5 chunks by cosine similarity retrieved from Qdrant with relevance scores.
10	■ QA Chain — cited answer	Mistral assembles retrieved chunks + question → answer with exact page citations. Temperature 0.1 for factual accuracy.

CREDENTIALS REQUIRED

CREDENTIAL	WHERE TO GET IT	n8n TYPE
Mistral API Key	console.mistral.ai → API Keys → Create new key	Mistral Cloud API
Qdrant Cloud URL + API Key	cloud.qdrant.io → Create Cluster (free tier: 1GB) → API Keys	Qdrant API

SAMPLE RUN — what this workflow processed and produced

Scenario	A small investment firm tracks 15 companies across quarterly and annual reports. Two analysts previously spent 3-4 hours each earnings season manually reading filings to populate a comparison spreadsheet.
Input received	Analyst dropped AAPL_Annual_FY2023.pdf into the watched folder. File watcher triggered ingestion automatically. 287 pages processed into 1,840 chunks in Qdrant in 4 minutes 12 seconds.
What the workflow found	Question 'What was gross margin FY2023 vs FY2022' matched top-5 chunks from pages 31, 38, 41 of the annual report. Retrieval latency: 380ms.
Output produced	Answer with exact figures and page citations returned in 2.1 seconds. Analyst confirmed all figures matched manual verification. Time saved per earnings season: approximately 6 hours across both analysts.

JSON: Build a Financial Documents Assistant using Qdrant and Mistral.ai.json	GitHub: trm-financial-rag-assistant	qdrant.tech/documentation/
---	--	--